TASKUP: GERENCIADOR DE TAREFAS

Thiago Vieira Alves Caldera
Frederico Barbosa da Silva
Gabriel Adriano Pontes
Olicio Souza Santos Junior
Sabrina Rayssa Teixeira

Professor: Ricardo Bicalho

RESUMO

Este artigo apresenta o TaskUp, um gerenciador de tarefas pessoais desenvolvido como projeto da unidade curricular Sistemas Distribuídos e Mobile. O objetivo é aplicar conceitos de desenvolvimento backend, modelagem de banco de dados e construção de APIs RESTful. O backend foi desenvolvido em Python, utilizando Django e Django REST Framework, com SQLite como banco de dados. O controle de versão é realizado com Git e GitHub. A API permite cadastrar, listar, consultar, atualizar e excluir tarefas, além de validar os dados recebidos. Os testes realizados por meio de requisições HTTP demonstraram o funcionamento das principais operações e das validações implementadas. Como próximas etapas, estão previstas a integração com as interfaces web e mobile e a publicação da aplicação em um ambiente de nuvem.

Palavras-chave: API RESTful; Python; Django; Django REST Framework; SQLite; gerenciamento de tarefas.

ABSTRACT

This paper presents TaskUp, a personal task manager developed for the Distributed and Mobile Systems course. The project aims to apply concepts of backend development, database modeling, and RESTful API design. The backend was developed in Python using Django and Django REST Framework, with SQLite as the database. Version control is managed with Git and GitHub. The API supports creating, listing, retrieving, updating, and deleting tasks, as well as validating incoming data. Manual HTTP request tests demonstrated the main operations and implemented validations. The next stages include integrating the web and mobile interfaces and deploying the application to a cloud environment.

Keywords: RESTful API; Python; Django; Django REST Framework; SQLite; task management.

1. INTRODUÇÃO

Um gerenciador de tarefas é uma aplicação adequada para praticar conceitos fundamentais de desenvolvimento de software. Nesse tipo de sistema, a interface do usuário atua como cliente, enquanto o servidor processa as requisições, aplica as regras da aplicação e armazena os dados. A comunicação entre essas partes pode ocorrer por meio de uma API, permitindo que diferentes interfaces utilizem os mesmos serviços.

O TaskUp foi desenvolvido na unidade curricular Sistemas Distribuídos e Mobile com o objetivo de organizar tarefas pessoais e aplicar conceitos de backend, banco de dados e APIs RESTful. Nesta etapa, o trabalho concentra-se na implementação e nos testes da API. A integração com as interfaces web e mobile e a publicação em nuvem são etapas previstas para a continuidade do projeto.

2. FUNDAMENTAÇÃO TEÓRICA E TECNOLOGIAS

2.1. API RESTful

REST é um estilo arquitetural descrito por Fielding (2000). Em uma API que segue esse estilo, os recursos são identificados por URIs e manipulados por meio de operações HTTP. Entre os métodos mais comuns estão GET, para consulta; POST, para criação; PUT, para atualização; e DELETE, para exclusão. As respostas HTTP incluem códigos de status que indicam o resultado da requisição, como 200 (sucesso), 201 (recurso criado), 400 (requisição inválida), 404 (recurso não encontrado) e 204 (operação concluída sem conteúdo de resposta). No TaskUp, os recursos principais são usuários e tarefas, acessados por rotas como /usuarios e /tarefas.

2.2. Tecnologias utilizadas

A linguagem Python e o framework Django foram utilizados na implementação do backend. O Django REST Framework complementa o Django com recursos para serialização de dados, validação e construção de endpoints de API. O SQLite foi escolhido como banco de dados, por armazenar os dados em um arquivo e não exigir a configuração de um servidor de banco de dados separado para o desenvolvimento local. Para as interfaces planejadas, estão previstas tecnologias web — HTML, CSS e JavaScript — e Flutter para a aplicação mobile. O Git é utilizado para controle de versão, enquanto o GitHub hospeda o repositório do projeto.

3. MODELAGEM E IMPLEMENTAÇÃO

3.1. Funcionalidades

As funcionalidades implementadas para o gerenciamento de tarefas estão resumidas na Tabela 1. Elas abrangem as operações básicas de criação, consulta, atualização e exclusão, conhecidas como CRUD, além da alteração do status da tarefa.

Tabela 1 — Funcionalidades do TaskUp.

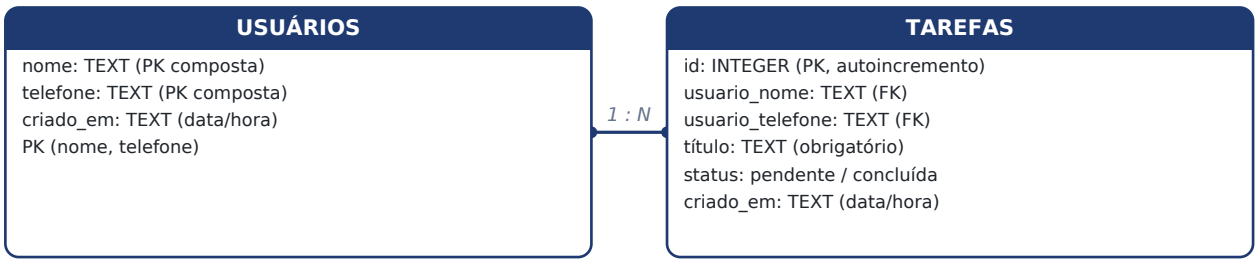
Código	Funcionalidade	Descrição
RF01	Cadastrar tarefa	Cria uma tarefa com título obrigatório e descrição opcional; o status inicial é pendente.
RF02	Listar tarefas	Retorna as tarefas cadastradas, das mais recentes para as mais antigas.
RF03	Visualizar detalhes	Retorna os dados de uma tarefa específica.
RF04	Editar tarefa	Permite alterar o título, a descrição ou o status da tarefa.
RF05	Marcar como concluída	Atualiza o status da tarefa para concluída.
RF06	Excluir tarefa	Remove a tarefa do banco de dados.

Fonte: Elaborado pelos autores (2026).

3.2. Modelo de dados

O modelo de dados descrito no projeto possui duas entidades relacionadas: usuários e tarefas. A entidade usuários armazena nome, telefone e data de criação, utilizando nome e telefone como chave primária composta. A entidade tarefas armazena identificador, título, descrição opcional, status e data de criação, além de referenciar o usuário por meio do nome e do telefone. Cada usuário pode estar associado a várias tarefas. A Figura 1 representa esse relacionamento conforme a modelagem apresentada pelos autores.

Figura 1 · Diagrama entidade-relacionamento do TaskUp.



Fonte: Elaborado pelos autores (2026).

3.3. API do backend

O backend utiliza modelos e migrações do Django para estruturar os dados, enquanto as views e os serializers do Django REST Framework são responsáveis pelo processamento das requisições e pela representação dos dados enviados e recebidos pela API. Conforme a especificação do projeto, os usuários são identificados por nome e telefone, e cada tarefa está vinculada a um usuário. A validação contempla campos obrigatórios, como o título, e os valores permitidos para o status da tarefa.

Tabela 2 — Principais endpoints da API.

Método	Rota	Função	Respostas esperadas
GET	/usuarios	Lista os usuários cadastrados.	200
POST	/usuarios	Cadastra um novo usuário.	201, 409
GET	/tarefas	Lista as tarefas.	200, 400
GET	/tarefas/:id	Consulta uma tarefa específica.	200, 404
POST	/tarefas	Cadastra uma nova tarefa.	201, 400, 404
PUT	/tarefas/:id	Atualiza os dados ou o status da tarefa.	200, 400, 404
DELETE	/tarefas/:id	Exclui uma tarefa.	204, 404

Fonte: Elaborado pelos autores (2026).

4. RESULTADOS PARCIAIS

4.1. Testes do backend

O backend foi executado localmente e avaliado por meio de requisições HTTP realizadas manualmente. Nos testes relatados pelos autores, o cadastro de uma tarefa válida retornou o código 201 e registrou o status inicial como pendente. Uma requisição de cadastro sem título retornou 400, indicando dados inválidos. A atualização do status para concluída foi realizada com sucesso, enquanto o envio de um status inválido retornou 400. A listagem retornou as tarefas cadastradas. Por fim, a exclusão retornou 204 e uma nova consulta ao recurso excluído retornou 404.

Os resultados indicam que as operações principais e algumas validações do backend foram exercitadas. Como os testes foram manuais, uma etapa futura pode incluir testes automatizados para ampliar a cobertura e facilitar a repetição das verificações.

4.2. Próximas etapas

As próximas etapas previstas são concluir as interfaces web, com HTML, CSS e JavaScript, e mobile, com Flutter; integrar os clientes à API; realizar testes funcionais com as interfaces conectadas ao servidor; e publicar a aplicação em um ambiente de nuvem. A conclusão dessas etapas permitirá avaliar o funcionamento do sistema de ponta a ponta.

5. CONSIDERAÇÕES FINAIS

O desenvolvimento do TaskUp permitiu aplicar conceitos de APIs RESTful, modelagem de dados e desenvolvimento backend com Python, Django e Django REST Framework. Conforme os testes descritos, a API oferece operações para cadastro, consulta, atualização e exclusão de tarefas, além de validações para os dados recebidos. O SQLite foi utilizado como banco de dados e o GitHub como ambiente de hospedagem do repositório e colaboração.

O projeto ainda prevê a integração com as interfaces web e mobile e a publicação em nuvem. Como possibilidades de evolução, destacam-se a implementação de autenticação de usuários, organização de tarefas por categorias, definição de prazos e envio de notificações. Essas funcionalidades podem ampliar a utilidade da aplicação em versões futuras.

6. REFERÊNCIAS

DJANGO. Django documentation. Disponível em: https://docs.djangoproject.com/. Acesso em: 12 out. 2026.

DJANGO REST FRAMEWORK. Django REST framework documentation. Disponível em: https://www.django-rest-framework.org/. Acesso em: 12 out. 2026.

FIELDING, Roy Thomas. Architectural styles and the design of network-based software architectures. 2000. Tese (Doutorado) — University of California, Irvine, 2000.

FLUTTER. Flutter documentation. Disponível em: https://docs.flutter.dev/. Acesso em: 12 out. 2026.

PYTHON. Python documentation. Disponível em: https://docs.python.org/3/. Acesso em: 12 out. 2026.

SQLITE. SQLite documentation. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 12 out. 2026.